RNA-seq Pipeline — Step-by-Step Explanation

Overview — What This Pipeline Does and Why

This pipeline identifies differentially expressed genes in Alzheimer's Disease and uses those patterns to build machine-learning classifiers, all from a single RNA-seq dataset of human brain tissue.

Dataset: GSE104704 — Mayo Clinic study, lateral temporal lobe (a region early affected in AD). 22 samples: 12 AD patients vs 10 age-matched Old Controls (ND). Published on GEO (Gene Expression Omnibus).

Why RNA-seq, not microarrays? Older studies (pre-2010) measured gene activity with microarrays — fluorescent probes on a chip. You could only measure genes the chip was designed for, signal had a ceiling, and noise was multiplicative. RNA-seq works differently: it sequences actual RNA molecules in the sample and counts how many reads map to each gene. This gives integer counts with no ceiling, the ability to discover novel transcripts, and a statistical distribution (negative binomial) that requires specialized tools — DESeq2, not a t-test.

The n=22 problem shapes every decision in this pipeline. With only 22 samples, standard practices break down:

- A simple 80/20 train-test split leaves only 4 test samples → accuracy on 4 samples is meaningless
- Variance estimates for 19,000+ genes from 22 samples are noisy → need DESeq2's shrinkage estimators
- Feature selection must be done inside each fold of cross-validation, never on the whole dataset

Steps	Purpose
1–2	Install packages, download GSE104704 from GEO
3–5	Load count matrix, extract metadata, fix sample ID matching
6–7	Build DESeq2 object, filter low-count genes

Steps	Purpose
8–9	Run DESeq2 (normalize → estimate variance → Wald test), shrink LFC
10	Volcano plot — visualize the DE landscape
11	GO and KEGG pathway enrichment
12	VST normalization — prepare data for ML
13–16	LOOCV loop — 22 folds × 5 classifiers, no data leakage
17	Compute accuracy/sensitivity/specificity, build comparison plots
18	Save all CSVs and PNG files

Steps 1–2: Package Installation and GEO Data Download

Step 1: Install packages. R does not ship with bioinformatics tools — they must be installed from two repositories: CRAN (general R packages) and Bioconductor (biology-specific tools). The `BiocManager::install()` function handles both.

Package	What it does
DESeq2	Differential expression analysis for RNA-seq count data
GEOquery	Downloads datasets from NCBI's Gene Expression Omnibus
clusterProfiler	Gene Ontology and KEGG pathway enrichment
org.Hs.eg.db	Maps human gene symbols to Entrez IDs (needed by clusterProfiler)
AnnotationDbi	Query gene annotation databases
ggplot2	Publication-quality plots
dplyr / tidyr	Data manipulation (filter, reshape, sort)
randomForest	Random forest classifier

Package	What it does
e1071	SVM (support vector machine)
glmnet	LASSO logistic regression
class	k-Nearest Neighbours (kNN)
naivebayes	Naive Bayes classifier
caret	confusionMatrix and preprocessing (center/scale)
apeglm / ashr	LFC shrinkage estimators for DESeq2

Step 2: Download GSE104704 from GEO. `getGEO("GSE104704")` connects to NCBI's GEO servers and returns the series matrix — a structured object containing both the count data and the sample metadata (phenotype table). `getGEOSuppFiles("GSE104704")` then downloads the supplementary raw count matrix file (a `.gz`-compressed tab-delimited file) which contains the integer read counts needed for DESeq2.

> Why not use the processed matrix from getGEO directly? GEO "series matrices" often store normalized or log-transformed values, not raw counts. DESeq2 requires raw integer counts — it does its own normalization internally. Using pre-normalized data with DESeq2 gives wrong results.

Steps 3–5: Count Matrix, Metadata, and the Sample Matching Fix

Step 3: Load the count matrix. The downloaded file is a tab-delimited text file with genes as rows and samples as columns. `read.delim()` reads it into R. The column names are sample IDs in the format `"{base}.RNA"` — for example, `"1200_RNA"` or `"AD_brain_RNA"`. We set gene symbols as row names and are left with a numeric matrix: 22 columns (samples) × ~20,000 rows (genes).

Step 4: Extract metadata from GEO. `pData(gset[[1]])` returns the phenotype table — one row per sample, with columns including `title`, `geo_accession`, tissue type, disease status, age, and so on. The `title` column uses a *different* naming format than the count matrix columns: GEO titles look like `"{base}.RNA.{Group}"` (e.g. `"1200_RNA.AD"` or `"1200_RNA.ND"`). This format mismatch would cause zero samples to match if used directly.

Step 5: The sample matching fix. This was the most technically tricky part of the pipeline. The fix is:

```
# Strip the last dot-segment from titles to get the base
meta$sample_id <- sub("\\.([^.]+)$", "", meta$title)
# meta$sample_id now = "1200_RNA" (matches the count matrix column names)

# Match count columns to metadata rows
common <- intersect(colnames(count_matrix), meta$sample_id)
count_matrix <- count_matrix[, common]
meta <- meta[match(common, meta$sample_id), ]
```

`sub("\\.([^.]+)$", "", ...)` is a regular expression that removes the last dot and everything after it. So `"1200_RNA.AD"` becomes `"1200_RNA"`, which matches the count matrix column `"1200_RNA"`. Without this fix, `intersect()` returns zero shared names and the entire analysis fails silently with an empty dataset.

After matching, we also check that the sample order in the count matrix matches the metadata row order — DESeq2 requires these to be aligned. We set `group` as a factor with levels `c("ND", "AD")` so that AD is the numerator in fold-change calculations (positive LFC = higher in AD).

Steps 6–7: Building the DESeq2 Object and Filtering Low-Count Genes

Step 6: Create the DESeqDataSet (DDS). DESeq2 needs a special container object that holds three things together: the count matrix, the sample metadata, and the experimental design formula.

```
dds <- DESeqDataSetFromMatrix(
  countData = count_matrix,   # integer matrix: genes x samples
  colData   = meta,           # metadata: one row per sample
  design    = ~ group         # the variable to test: AD vs ND
)
```

The `design = ~ group` formula tells DESeq2 to test for differences between the levels of the `group` variable. DESeq2 uses this formula to build a design matrix — the same concept as linear regression, where each sample gets a 0 or 1 indicating its group. The model then estimates how much each gene's expression differs between groups.

Step 7: Filter low-count genes. Of the ~19,000 genes in the matrix, many have near-zero counts across all samples. These genes:

1. Carry no statistical information about AD vs ND
2. Inflate the multiple testing burden (more tests = stricter FDR correction)
3. Have unreliable variance estimates in small datasets

We keep only genes where at least `min.samples` samples have a count ⩾ 10 (where `min.samples` = the size of the smaller group, 10 ND samples). This is the standard "independent filtering" threshold. After filtering, we went from ~19,130 to ~12,000–15,000 expressed genes (the exact number depends on the cutoff). DESeq2 also applies its own internal independent filtering during results extraction, further reducing the gene list.

> **Why integers?** DESeq2's negative binomial model requires raw counts. If you give it log-transformed or normalized values, the model is mis-specified and p-values will be wrong. This is a common mistake when moving from microarray analysis (where you work with log2 intensities) to RNA-seq.

Step 8: Running DESeq2 — Three Internal Steps in One Function

`dds <- DESeq(dds)` looks like one line of code, but it runs three distinct statistical steps under the hood:

Step 8a: Size factor normalization. Different samples are sequenced to different depths — one sample might have 20 million reads mapped and another only 15 million. A gene with 1,000 counts in the deep-sequenced sample isn't more expressed than the same gene with 800 counts in the shallow sample. DESeq2 calculates a *size factor* for each sample using the median-of-ratios method: for each gene, it computes the geometric mean count across all samples, then divides each sample's count by that mean. The median of these ratios across genes is the size factor. This is more robust than simple total-count normalization because it ignores genes with extreme outlier counts.

Step 8b: Dispersion estimation. RNA-seq counts have more variance than a Poisson distribution predicts. This extra variance is called *overdispersion* and is captured by the dispersion parameter of the negative binomial model. With only 22 samples, per-gene dispersion estimates are noisy — a gene observed in only 22 replicates has a poor estimate of its true variance. DESeq2 solves this by *sharing information across genes* using an empirical Bayes shrinkage approach: it fits a smooth curve to the per-gene

dispersions as a function of mean expression, then shrinks each gene's estimate toward this curve. Genes that are closer to the curve (typical behavior) get pulled toward it more strongly; genes with unusual dispersion retain more of their observed estimate.

Step 8c: Wald test. With normalized counts and dispersion estimates in place, DESeq2 fits a generalized linear model (GLM) for each gene and tests the coefficient for `group` using a Wald test (coefficient ÷ standard error, compared to a standard normal distribution). This gives a p-value for every gene. DESeq2 then applies Benjamini-Hochberg correction to control the False Discovery Rate (FDR) — the adjusted p-value `padj` is what we filter on.

Results after Step 8: 1,825 genes with padj < 0.05 out of ~12,000–15,000 tested. That's nearly 10% of expressed genes significantly changed in AD temporal cortex — a large effect that reflects the profound biology of late-stage AD.

Step 9: LFC Shrinkage (ashr) and Results Extraction

Why shrink log2 fold changes? The raw LFC (log2FoldChange) from the Wald test is noisy for genes with low counts. A gene expressed at an average of 1 count in AD and 2 counts in ND has a raw LFC of −1.0, but this estimate has enormous uncertainty — one extra read changes everything. Without correction, these low-count genes often appear as the "most differentially expressed" simply because their estimates are most variable, not because they have strong biological signals.

The ashr approach. We apply LFC shrinkage using the `ashr` method (`lfcShrink(dds, contrast, type="ashr")`). The `ashr` package fits an adaptive shrinkage prior: it looks at the distribution of all LFC estimates across genes and infers how much noise is likely, then uses that prior to pull uncertain estimates toward zero. Genes with strong statistical evidence (high counts, large LFC, small SE) are pulled minimally. Genes with weak evidence (low counts, small LFC) are pulled strongly toward zero. The result is a *more honest* set of effect sizes: the top-ranked genes are ones where there is genuine evidence of a large effect, not just random noise amplified by low counts.

Results extraction: `results(dds)` extracts the full table including:

Column	Meaning
baseMean	Average normalized count across all 22 samples
log2FoldChange	Shrunken log2(AD/ND) — positive = higher in AD

Column	Meaning
lfcSE	Standard error of the LFC estimate
stat	Wald statistic (LFC / lfcSE)
pvalue	Raw p-value from the Wald test
padj	Benjamini-Hochberg adjusted p-value (FDR)

We filter to `padj < 0.05` and examine the top hits. **VGF** (LFC −2.14, padj < 0.001) is the most significantly downregulated gene — a neuropeptide precursor already known to decline in AD cerebrospinal fluid and serve as a biomarker candidate. **KLF15** (LFC +1.14) is the most upregulated, a transcription factor involved in metabolic stress responses.

Step 10: Volcano Plot — Reading the DE Landscape

A volcano plot is the standard visualization for differential expression results. It places every gene as a point in a two-dimensional space:

- **X-axis**: log2 fold change (LFC) — how much more or less the gene is expressed in AD vs ND. Positive = higher in AD; negative = lower in AD.
- **Y-axis**: −log10(padj) — statistical significance. Higher = more significant. A gene with padj = 0.05 sits at y = 1.3; padj = 0.001 sits at y = 3.

How to read the shape: The plot looks like a volcano because most genes cluster near (0, 0) — unchanged and not significant. The two "wings" at the top-left and top-right are the significant DE genes: high up (significant) and far left (down in AD) or far right (up in AD). The "neck" of the volcano — genes that are statistically significant but have small fold changes — may be biologically real but unlikely to be useful as individual biomarkers.

What we see in GSE104704:

There are more significantly downregulated genes (left wing) than upregulated (right wing) in AD temporal cortex, consistent with the known neuronal loss and synaptic silencing in Alzheimer's disease. The top genes labeled on the plot include:

- **VGF** (far left, high y) — the most significantly downregulated gene. VGF is a neurotrophin-inducible protein whose expression is driven by BDNF. Its loss in AD has been replicated across multiple brain regions and body fluids, making it one of the best-validated DE genes in AD research.

- **KLF15** (far right) — upregulated. A transcription factor involved in metabolic and stress responses in neurons.

Coloring convention: We used three colors: grey (not significant), blue (|LFC| > 1 but padj ⩾ 0.05), and red (padj < 0.05). Red points are the 1,825 genes we carry forward. The blue points represent genes with large fold changes that didn't survive FDR correction — likely because they're expressed in few samples or have high within-group variance.

Steps 11–12: Pathway Enrichment and VST Normalization for ML

Step 11: Pathway enrichment (clusterProfiler). Once we have a list of DE genes, we ask: do these genes belong to known biological pathways more often than expected by chance? This is enrichment analysis.

We convert gene symbols to Entrez IDs (the identifier system KEGG and GO databases use), then run:

- `enrichGO()` — tests over-representation in Gene Ontology Biological Process (GO:BP) terms. GO:BP describes biological processes like "synaptic transmission" or "apoptotic process" hierarchically.
- `enrichKEGG()` — tests over-representation in KEGG pathway maps (more specific: e.g. "Alzheimer disease" pathway, "Long-term potentiation").

Both use a hypergeometric test: given that 1,825 of 15,000 expressed genes are DE, what is the probability that ⩾k of the n genes in pathway X are in our DE list by chance? The resulting dotplots show pathway name, gene ratio (what fraction of the pathway's genes are DE), and adjusted p-value.

Step 12: VST normalization for ML. Raw counts cannot go into ML models directly for two reasons:

1. **Scale**: a highly expressed gene might have counts in the thousands while a lowly expressed gene has counts < 10. Random forest and kNN treat these on the same numeric scale, so the high-count gene dominates every split/distance.
2. **Heteroscedasticity**: the variance of a count is proportional to its mean (larger mean = larger variance). Most ML models assume roughly equal variance across features.

`vst(dds, blind=FALSE)` applies Variance Stabilizing Transformation, which:

- Takes log2-scale of the counts, with a correction for the mean-variance relationship estimated from this dataset

- Produces values where variance is approximately constant across the range of expression
- Uses `blind=FALSE` to use the fitted dispersion estimates from DESeq2 (more accurate than a blind fit)

The result is a matrix of 22 samples × all expressed genes in roughly comparable, stabilized units. We transpose it to `samples × genes` format for ML (R's convention: ML expects rows = samples, columns = features).

Steps 13–16: The LOOCV Loop — Why, How, and What Each Model Does

Why LOOCV and not an 80/20 split? With n=22, an 80/20 split assigns 17 samples to training and 4 to testing. Accuracy measured on 4 samples — 2 AD and 2 ND — is statistically meaningless: a single prediction wrong changes accuracy from 100% to 75%. LOOCV (Leave-One-Out Cross-Validation) solves this by using every sample as a test sample exactly once.

In each fold:

1. Hold out sample `i` as the test set
2. Train all 5 models on the remaining 21 samples
3. Predict the label of sample `i`
4. Record whether the prediction was correct

Repeat for all 22 samples. Final accuracy = number of correct predictions / 22. This gives you 22 independent test points, each from a model that has never seen that sample.

The no-leakage rule. A critical mistake in many published ML papers is performing feature selection on the entire dataset before cross-validation. If you first select the top 500 most variable genes from all 22 samples, then run LOOCV, the test sample's data influenced which genes were selected — this is data leakage, and it inflates accuracy. In this pipeline, feature selection and scaling happen *inside* each fold:

```
# Inside fold i:
gene_vars <- apply(X_train_full, 2, var)   # variance on 21 training samples
ONLY
top500 <- names(sort(gene_vars, decreasing=TRUE)[1:500])
X_tr <- as.data.frame(X_train_full[, top500])
X_te <- as.data.frame(X_test_one[, top500, drop=FALSE])

pp <- preProcess(X_tr, method=c("center","scale"))  # fit scaler on 21
samples
X_tr <- predict(pp, X_tr)   # scale training set
X_te <- predict(pp, X_te)   # apply SAME scaler to test sample
```

The test sample is treated as completely unknown: it cannot influence which genes are selected or how the data is scaled.

The five models:

Model	How it works	Why included
Random Forest (RF)	Builds 300 decision trees, each on a random subset of features. Predicts by majority vote.	Handles correlated features well; robust to outliers; provides feature importance
SVM (radial kernel)	Finds the maximum-margin hyperplane separating AD from ND in a high-dimensional kernel space	Strong performance in high-p, low-n settings; theoretically sound
LASSO	Logistic regression with L1 penalty that forces most gene coefficients to exactly zero, retaining only the most predictive genes	Automatic feature selection; interpretable coefficients
KNN (k=3)	Predicts based on the 3 nearest neighbors in the 500-gene feature space	Simple baseline; sensitive to the curse of dimensionality
Naive Bayes	Assumes features are conditionally independent given the class; computes class probabilities from each gene's distribution	Fast; good benchmark; performs surprisingly well in genomics

Key R bug fixed in this pipeline: `X_test_one[, top500]` without `drop=FALSE` on a 1-row matrix collapses it to a named vector, losing its dimensions. All subsequent functions that expect a 2D matrix fail with `"incorrect number of dimensions"`. The fix is `as.data.frame(X_test_one[, top500, drop=FALSE])` which preserves the matrix structure.

Step 17: Metrics — Accuracy, Sensitivity, Specificity, and What the Results Mean

The three metrics explained:

- **Accuracy** = (TP + TN) / (TP + TN + FP + FN) × 100. The fraction of all 22 samples correctly classified. Simple but can be misleading when classes are imbalanced (12 AD, 10 ND here).
- **Sensitivity** (= Recall = True Positive Rate) = TP / (TP + FN) × 100. Of the 12 AD patients, how many were correctly identified as AD? A model that calls everyone "AD" gets 100% sensitivity but 0% specificity.
- **Specificity** (= True Negative Rate) = TN / (TN + FP) × 100. Of the 10 controls, how many were correctly identified as ND? Important for not over-diagnosing.

For a disease classifier, both sensitivity and specificity matter. High sensitivity catches more cases (fewer missed AD patients); high specificity reduces false alarms (fewer healthy people told they have AD).

The metrics fix. The initial code used `caret::confusionMatrix()` and extracted metrics as `cm$overall["Accuracy"]`. The problem: this returns a *named numeric* `c(Accuracy=0.955)`, and when you wrap it in `c(Accuracy = cm$overall["Accuracy"])`, R creates a doubly-named element `"Accuracy.Accuracy"`. Indexing `rf_metrics["Accuracy"]` then returns NA because the name doesn't match. Fixed by computing metrics directly from TP/TN/FP/FN counts:

```
TP <- sum(preds == "AD" & truth == "AD")
TN <- sum(preds == "ND" & truth == "ND")
FP <- sum(preds == "AD" & truth == "ND")
FN <- sum(preds == "ND" & truth == "AD")
acc  <- round((TP + TN) / length(truth) * 100, 1)
sens <- round(TP / (TP + FN) * 100, 1)
spec <- round(TN / (TN + FP) * 100, 1)
```

Final LOOCV results (n=22, no data leakage):

Model	Accuracy	Sensitivity	Specificity	Interpretation
LASSO	100%	100%	100%	Perfect — credible given strong DE signal
Random Forest	95.5%	91.7%	100%	1 AD misclassified as ND
SVM	81.8%	83.3%	80%	Solid; some boundary overlap
KNN (k=3)	68.2%	91.7%	40%	Finds AD but misses 6 of 10 controls
Naive Bayes	63.6%	58.3%	70%	Worse than random on sensitivity

Is LASSO 100% believable? Yes, for this dataset. With 1,825 significantly DE genes (nearly 10% of the transcriptome), the AD vs ND separation in lateral temporal cortex is biologically extreme — this is late-stage disease. LASSO is especially suited here because it performs feature selection automatically: in each fold it identifies the handful of most discriminative genes from the 500 candidates and uses only those. Each of the 22 test samples was predicted by a model trained on the other 21 — it was never "seen" during training.

KNN's low specificity (40%) is expected. With 500 features and only 21 training points, Euclidean distance is dominated by noise — this is the "curse of dimensionality". The 12 AD majority class dominates the k=3 vote for borderline samples.

Pathway Enrichment: GO and KEGG with clusterProfiler

What enrichment analysis asks: given a list of DE genes, do they cluster in specific biological pathways more than you'd expect by chance? For example, if 50 of your 1,825 DE genes belong to the "synaptic vesicle cycle" pathway, but only 60 genes in that pathway are even expressed in this dataset, then 50/60 = 83% of the pathway is changed — that is very unlikely by chance.

Gene Ontology (GO) enrichment. The Gene Ontology is a hierarchical vocabulary of biological terms organized into three branches: Biological Process (BP), Molecular Function (MF), and Cellular Component (CC). We focus on GO:BP, which describes higher-level processes like:

- "Neurotransmitter secretion" — synaptic processes disrupted in AD

- "Regulation of apoptotic process" — cell death pathways
- "Axon development" — neuronal structural changes

`enrichGO(gene, OrgDb=org.Hs.eg.db, ont="BP", pAdjustMethod="BH", pvalueCutoff=0.05)` tests all GO:BP terms using a hypergeometric test and BH correction. The gene IDs must be Entrez integers — hence the `bitr()` conversion from gene symbols first.

KEGG pathway enrichment. KEGG (Kyoto Encyclopedia of Genes and Genomes) contains curated maps of metabolic and signaling pathways. Unlike GO's tree structure, KEGG pathways are drawn as actual molecular interaction maps. For AD research, the most informative KEGG results are:

- **hsa05010 — Alzheimer disease**: the canonical AD pathway (amyloid, tau, mitochondria, apoptosis)
- **hsa04724 — Glutamatergic synapse**: synaptic transmission (early-affected in AD)
- **hsa04728 — Dopaminergic synapse**: reward and motor circuits
- **hsa04015 — Rap1 signaling**: neuronal plasticity and survival

Visualization fix. The `dotplot()` function from enrichplot had a bug in some versions: it internally calls `unique()` on a data frame, which fails because `unique()` only accepts vectors in that context. The fix is to bypass the enrichplot function and build the dotplot manually using the enrichment result's `@result` slot:

```
kegg_df <- as.data.frame(kegg_results@result) %>%
  filter(p.adjust < 0.05) %>%
  arrange(p.adjust) %>%
  head(15)

ggplot(kegg_df, aes(x=GeneRatio_num, y=reorder(Description, -p.adjust),
                    size=Count, color=p.adjust)) +
  geom_point() +
  scale_color_gradient(low="red", high="blue") + ...
```

The `GeneRatio` column is a string like `"25/400"` — we convert it to a number by parsing the two parts and dividing, then use it as the x-axis. This approach works regardless of enrichplot version.

Final Results, Limitations, and Next Steps

What this analysis demonstrates:

Using 22 post-mortem human brain samples (lateral temporal lobe), this pipeline shows that:

1. **1,825 genes** are significantly differentially expressed in AD vs age-matched controls (padj < 0.05, DESeq2 LOOCV)
2. **VGF downregulation** is the strongest single-gene signal (LFC −2.14) — a neuropeptide already validated as an AD biomarker candidate in independent cohorts
3. **Synaptic and metabolic pathways** are the most enriched: glutamatergic synapse, Alzheimer disease pathway, neurotrophin signaling
4. **LASSO achieves 100% LOOCV accuracy** (22/22 correctly classified) with no data leakage — Random Forest achieves 95.5% — demonstrating that gene expression profiles in temporal cortex are sufficient to discriminate AD from controls at the individual sample level

Limitations:

Limitation	Why it matters
n=22 is very small	LOOCV results, especially 100% accuracy, may not generalize to new cohorts
Post-mortem tissue only	Cannot be used directly as a blood/CSF biomarker — brain is not accessible in living patients
Single brain region	Lateral temporal lobe; results may differ in frontal cortex, hippocampus, or cerebellum
No replication cohort	The 100% accuracy should be validated in an independent dataset (e.g. GSE5281 or ROSMAP)
Bulk RNA-seq	Averages across all cell types; neuronal loss in AD means some "DE genes" reflect cell type proportion changes rather than per-cell expression changes
Late-stage AD only	All 12 AD patients had confirmed pathology; early-stage AD or MCI may show weaker signals

What to do next:

- **Replicate in an independent cohort** — GSE5281 (Affymetrix, multiple brain regions) or ROSMAP (RNA-seq, dorsolateral prefrontal cortex) are good starting points
- **Single-cell RNA-seq** — would resolve which cell types drive the VGF and synaptic gene downregulation
- **Protein validation** — confirm VGF protein levels by ELISA or immunohistochemistry in matched tissue
- **CSF/plasma biomarker check** — compare the top DE genes against known CSF AD biomarkers (VGF is already there; check others in AlzBiomarker database)
- **Extend to MCI samples** — test whether the classifier can detect pre-AD changes (mild cognitive impairment), which is clinically more valuable
- **Regularized multi-gene signature** — the LASSO coefficients from a full-data model identify the minimal gene set; check whether 10–20 genes suffice for classification